TECNOLÓGICO DE ESTUDIOS SUPERIORES DE IXTAPALUCA

INGENIERÍA EN SISTEMAS COMPUTACIONALES

REDES NEURONALES ARTIFICIALES

REPORTE DE RESULTADOS — ACTIVIDAD 2

Docente:

Ing. Esteban Ramírez de la Rosa

Integrantes:

Espitia Zavala Gerardo

Olvera Rodríguez Bruno

Semestre: 9.º Grupo: 1951

Octubre de 2026

1. Introducción y objetivo

Se implementó en Python una red neuronal multicapa que aprende la compuerta XOR. La actividad permite observar directamente el paso hacia adelante, el cálculo de la pérdida, la retropropagación y la actualización de pesos, además de comparar la convergencia con diferentes tasas de aprendizaje.

2. Metodología y herramientas

Herramientas: Python, NumPy (cálculo matricial), Tkinter (interfaz) y Matplotlib (gráficas). Se eligió una red totalmente conectada 2–2–1: dos entradas binarias, dos neuronas ocultas y una salida. Una sola neurona sin capas ocultas no separa XOR; dos unidades ocultas permiten construir fronteras no lineales. Se aplicó sigmoide en ambas capas por su no linealidad y derivada utilizable en retropropagación. Los pesos y sesgos son entrenables; la pérdida es entropía cruzada binaria (BCE) y se optimiza con descenso por gradiente de lote completo.

En cada época, la red ejecuta: (1) $z_1 = XW_1 + b_1$ y $h = \sigma(z_1)$; (2) $z_2 = hW_2 + b_2$ y $\hat{y} = \sigma(z_2)$; (3) pérdida BCE; (4) gradientes mediante regla de la cadena y (5) actualización $W \leftarrow W - \eta\nabla W$, $b \leftarrow b - \eta\nabla b$. Los datos son (0,0)→0, (0,1)→1, (1,0)→1 y (1,1)→0.

Se realizaron dos entrenamientos desde exactamente los mismos pesos iniciales (semilla aleatoria 17), durante 10,000 épocas, modificando únicamente la tasa de aprendizaje: η=0.1 y η=1.0. El umbral de decisión fue 0.5.

3. Resultados de los experimentos

Indicador	**Experimento 1**	**Experimento 2**
Tasa de aprendizaje	0.1	1.0
Épocas	10,000	10,000
Pérdida BCE final	0.0126158	0.0009507
Exactitud sobre 4 casos	100 %	100 %

Las probabilidades finales en el orden (0,0), (0,1), (1,0), (1,1) fueron: η=0.1 → [0.012254, 0.983709, 0.988766, 0.010357]; η=1.0 → [0.000910, 0.998746, 0.999154, 0.000790]. Todas las clases resultaron correctas al aplicar el umbral de 0.5.

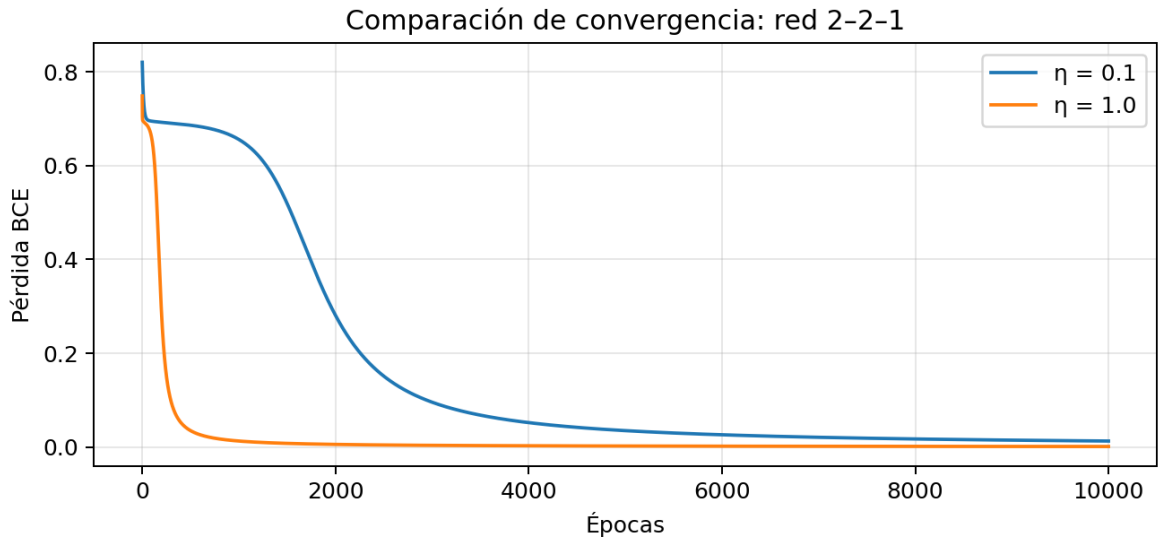


Figura 1. Curvas de pérdida BCE por época con dos tasas de aprendizaje; resultados generados por el código Python.

4. Análisis de convergencia

Ambos entrenamientos convergieron hacia soluciones que clasifican correctamente las cuatro entradas. Con η=1.0 la reducción de pérdida fue más rápida y el valor final menor bajo estas condiciones. Una tasa mayor no garantiza siempre mejores resultados: en otros problemas puede causar oscilaciones o inestabilidad. La arquitectura multicapa sí puede representar XOR, a diferencia del perceptrón simple.

5. Sobreajuste y limitaciones

No se puede diagnosticar sobreajuste rigurosamente: XOR contiene cuatro patrones y no se reservó prueba independiente. Tampoco hay underfitting al

finalizar, pues ambos modelos clasifican los cuatro casos. La exactitud del 100 % solo corresponde a estas entradas conocidas, no a datos nuevos.

6. Evidencia del funcionamiento y colaboración

El simulador permite seleccionar una entrada XOR y observar activaciones, pesos, sesgos y predicciones. El motor red_xor.py realiza los cálculos; simulador_xor.py presenta la interfaz y exporta las curvas. Se adjunta el archivo .py comentado y las imágenes PNG generadas. Registro de colaboración:

Espitia Zavala Gerardo: Participación en la implementación y revisión del funcionamiento de la red neuronal multicapa 2–2–1, verificando el procesamiento de las entradas XOR, las funciones de activación, el cálculo del error y la actualización de pesos mediante retropropagación. Asimismo, participación en las pruebas de funcionamiento para comprobar que las predicciones correspondieran con los resultados esperados de la compuerta XOR.

Olvera Rodríguez Bruno: Participación en la revisión de la interfaz gráfica del simulador y en la realización de los experimentos con tasas de aprendizaje de 0.1 y 1.0. Análisis de las gráficas de pérdida frente a las épocas, comparación de los resultados obtenidos y revisión de la documentación correspondiente.

7. Conclusiones individuales

Espitia Zavala Gerardo: la capa oculta hace posible representar XOR, a diferencia de un perceptrón simple. Observar las activaciones y los gradientes permite relacionar cada actualización con el descenso de la pérdida.

Olvera Rodríguez Bruno: al usar los mismos pesos iniciales, la comparación aísla el efecto de la tasa de aprendizaje sobre la convergencia. Aprender las cuatro entradas de XOR no demuestra generalización a datos nuevos.

8. Referencias y apoyo tecnológico

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536. https://doi.org/10.1038/323533a0

OpenAI. (2026). ChatGPT [Modelo de lenguaje]. https://chatgpt.com/ (preparación de materiales).